

How To Present Performance Results

Santhoshi Karri
Vijayakumar (Rittu) Rittambhra Rani
Carl Chang

Relationship between Latency and Throughput?

- Latency and throughput work together to deliver high network connectivity and performance.
- As both impact the transmission of data packets they also affect one another

“Many eggs can boil in parallel in a big pot, but a single egg still takes its full time”

How can you Improve Latency and Throughput?

- To improve latency, by shorten the propagation between the source and destination, reducing processing delays and minimizing round trips.
- To improve throughput, by increasing the overall network Bandwidth, adding parallelism, improving resource utilization and eliminating bottlenecks.

Some suggestions to improve latency and throughput together

- Caching
- Transport protocols
- Quality of Service

Measuring Throughput or Latency?

```
1  template <typename T>
2  void match_multiple_smh(T & smh, std::vector<u8 *> & buffers, std::vector<size_t> & lengths,
3                          std::vector<u32> & results) {
4      u32 i = 0;
5      #ifndef NO_UNROLL
6          for (; i+7 < buffers.size(); i+=8) {
7              results[i+0] = smh.match(buffers[i+0], lengths[i+0]); LFENCE
8              results[i+1] = smh.match(buffers[i+1], lengths[i+1]); LFENCE
9              results[i+2] = smh.match(buffers[i+2], lengths[i+2]); LFENCE
10             results[i+3] = smh.match(buffers[i+3], lengths[i+3]); LFENCE
11             results[i+4] = smh.match(buffers[i+4], lengths[i+4]); LFENCE
12             results[i+5] = smh.match(buffers[i+5], lengths[i+5]); LFENCE
13             results[i+6] = smh.match(buffers[i+6], lengths[i+6]); LFENCE
14             results[i+7] = smh.match(buffers[i+7], lengths[i+7]); LFENCE
15         }
16     #endif
17     for (; i < buffers.size(); ++i) {
18         results[i] = smh.match(buffers[i], lengths[i]); LFENCE
19     }
20 }
```

Each `smh.match()` call operates on different buffers, so the CPU can execute multiple operations simultaneously using out-of-order execution.

Measuring Throughput or Latency?

```
1  template <typename T>
2  void match_multiple_smh_latency_test(T & smh, std::vector<u8 *> & buffers, std::vector<
3      std::vector<u32> & results) {
4      u32 i = 0;
5      u32 tmp = 0;
6      #ifndef NO_UNROLL
7          // NOTE: experimental code only. Note that the addition of 'tmp' – being the id
8          // match – could take us RIGHT outside our buffer if we actually matched someth
9          // in this particular run, but so it goes. Saner would be to build up an all-ze
10         for (; i+7 < buffers.size(); i+=8) {
11             tmp = results[i+0] = smh.match(buffers[i+0 + tmp], lengths[i+0] + tmp); LFEI
12             tmp = results[i+1] = smh.match(buffers[i+1 + tmp], lengths[i+1] + tmp); LFEI
13             tmp = results[i+2] = smh.match(buffers[i+2 + tmp], lengths[i+2] + tmp); LFEI
14             tmp = results[i+3] = smh.match(buffers[i+3 + tmp], lengths[i+3] + tmp); LFEI
15             tmp = results[i+4] = smh.match(buffers[i+4 + tmp], lengths[i+4] + tmp); LFEI
16             tmp = results[i+5] = smh.match(buffers[i+5 + tmp], lengths[i+5] + tmp); LFEI
17             tmp = results[i+6] = smh.match(buffers[i+6 + tmp], lengths[i+6] + tmp); LFEI
18             tmp = results[i+7] = smh.match(buffers[i+7 + tmp], lengths[i+7] + tmp); LFEI
19         }
20     #endif
21     for (; i < buffers.size(); ++i) {
22         tmp = results[i] = smh.match(buffers[i + tmp], lengths[i + tmp]); LFENCE
23     }
24 }
```

Each iteration uses tmp, which depends on the previous result, preventing instruction overlap.

the benchmark env.

A NUMBER IS MEANINGLESS WITHOUT
CONTEXT

Hardware Required:

- Specific Microarchitecture (e.g., *i9-12900K*, not just "i7").
- Memory Config (*DDR5-4800*).

Software Required:

- Kernel Version (`uname -r`).
- Compiler & Flags (`gcc -O3` vs `-O2`).

System State:

- Turbo Boost? (Frequency noise).
- Background Daemons? (Dropbox, Chrome).

```
typerubu@RittuPREDATOR:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian

CPU(s):                 32
On-line CPU(s) list:   0-31
Vendor ID:              GenuineIntel
Model name:             13th Gen Intel(R) Core(TM) i9-13900HX
CPU family:             6
Model:                 183
Thread(s) per core:    2
Core(s) per socket:    16
Socket(s):              1
Stepping:               1
BogoMIPS:               4838.40
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse s
se2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonst
op_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe p
opcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd i
brs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi
2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni
vmmi umip waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear serialize flush_l
1d arch_capabilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:      Microsoft
Virtualization type:    full

Caches (sum of all):
L1d:                    768 KiB (16 instances)
L1i:                    512 KiB (16 instances)
L2:                     32 MiB (16 instances)
L3:                     36 MiB (1 instance)

NUMA:
NUMA node(s):           1
NUMA node0 CPU(s):     0-31

Vulnerabilities:
Gather data sampling:   Not affected
Itlb multihit:          Not affected
L1tf:                   Not affected
Mds:                    Not affected
Meltdown:               Not affected
Mmio stale data:        Not affected
Reg file data sampling: Vulnerable: No microcode
Retbleed:               Mitigation; Enhanced IBRS
Spec rstack overflow:   Not affected
Spec store bypass:      Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:             Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:             Mitigation; Enhanced / Automatic IBRS; IBPB conditional; RSB filling; PBSRB-eIBRS SW sequence;
                        BHI BHI_DIS_S
Srbds:                  Not affected
Tsx async abort:        Not affected
```

wall clock vs monotonic clock

wall clock ❌

```
time.time(), gettimeofday()
```

Purpose: Telling humans "It is 12:00 PM."

Mechanism: Syncs with the Internet (NTP).

Fatal Flaw: It can jump backwards.

Scenario: If your internal clock is fast (12:01) and the internet server says "No, it's 12:00," your OS resets the time back by one minute.

Result: Your benchmark duration becomes **negative** (-59 seconds). This destroys your data.

monotonic clock ✅

```
std::chrono::steady_clock, CLOCK_MONOTONIC.
```

Purpose: Measuring **Intervals** (Duration).

Mechanism: Counts hardware "ticks" from the moment the system boots up.

Guarantee: It never decreases, only goes up.

Resolution: Nanoseconds (vs Milliseconds for Wall Clock).

Why measure performance and present results

Correctness is not enough. Being efficient is also important.

Performance depends on many external factors. Numbers are meaningless without context.

Runtime depends on many factors.

Hardware configuration

CPU: executes instructions like math operations, memory access, etc, basically controls everything

Clock speed: cycles per second, directly increasing code performance, e.g. 3 GHz CPU means 3 billion cycles per second, the more it has the less time it would take

Core number: really important for parallel program, divide works among cores, e.g. 8 cores means each does $1/8$ of the total work

Cache: temporary storage for frequent access

RAM: slow memory, storage for program data, determines how much data and how many programs your computer can keep readily accessible for the CPU.

Key point: hardware determines performance capability

Software configuration

Compiler: converts code into machine instructions, managing things such as instructions, registers, loops, can be inefficient without proper optimizations such as remove unnecessary instructions, reduce memory access, uses less CPU cycles

Operating system: handles stuff like memory management, scheduling, thread, system call, different OS handle these differently

Libraries: libraries themselves provides optimized algorithms, different libraries worked differently, e.g. dynamic libraries, static libraries

Key point: software determines how efficiently hardware is used

System state

Would affect performance through a few ways

Background programs: the less the best

Cache interference: cache races, multiple processes compete for limited shared cache capacity, overwrite and reload of cache

System load causes inconsistent results: resource contention, forcing processes to wait for CPU time, if system load changes then available CPU change

Multiple runs is needed: dealing with noise sources such as OS scheduling, cache state, background activity

Key point: why they matter

Hardware configuration, software configuration and system state must be reported.

Hardware determines how fast instructions can execute.

Software determines how efficiently hardware is used.

System state affects measurement accuracy.

Without them, performance results might not be accurate enough and cannot be properly interpreted, which makes the comparisons unfair.

choosing the right mean (statistical competence)

Mean Type	Formula	When to Use	Example
Arithmetic	Sum / n	Pure Time or Latency	"Average Latency: 5ms"
Harmonic	$n / \sum(1/x)$	Rates (Speed, Throughput)	"Bandwidth: 100 MB/s"
Geometric	$\sqrt[n]{\prod x}$	Ratios or Speedups	"System A is 1.5x faster"

the “overlap” rule: confidence intervals

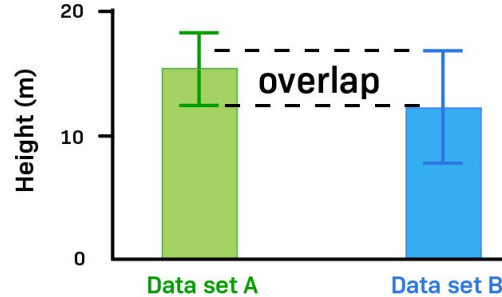
illusion: A single number implies 100% certainty.

reality: Every system has noise.

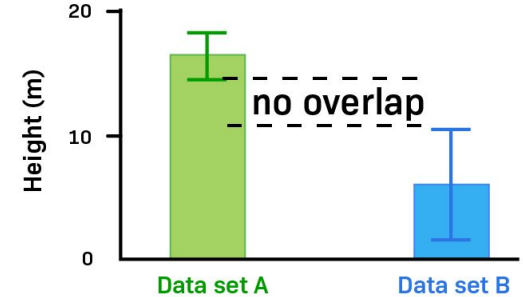
requirement: Always plot **95% Confidence Intervals** (Error Bars).

Rule of thumb:

- If bars **Overlap** → Statistically Indistinguishable (Tie).
- If bars **Separate** → Valid Win.



When standard deviation errors bars overlap a lot, it can be a clue that the difference *may not be* statistically significant. However you must perform statistical tests to draw a conclusion.



When standard deviation errors bars do not overlap, it can be a clue that the difference *may be* statistically significant, but we cannot be sure without performing statistical tests to draw a conclusion.

Flame Graphs - <https://www.brendangregg.com/flamegraphs.html>

Why do we need Flame Graph?

- Where is CPU time going?
- Which function is taking the most time?
- What part of the code should we optimize?

Flame Graph

A flame graph is a visualization of many stack traces aggregated together. It was originally used to show what is “hot” on-CPU.

Each box represents a function in the program.

The width of a box shows how much time was spent there.

The vertical direction shows the call stack (who called whom).

How to make flame graph (terminal commands)

Do this step by step:

(Skip the following line if on sunlab)

```
sudo apt install linux-tools-common linux-tools-generic linux-tools-$(uname -r)
```

```
git clone https://github.com/brendangregg/FlameGraph.git
```

```
g++ -O2 -g your_program.cpp -o program
```

```
perf record -F 99 -g ./your_program (this would output perf.data)
```

```
perf script > out.perf
```

```
./FlameGraph/stackcollapse-perf.pl out.perf > out.folded
```

```
./FlameGraph/flamegraph.pl out.folded > flamegraph.svg
```

lastly download flamegraph.svg or use firefox flamegraph.svg if you have firefox

Day 2

How to Measure, Present, and compare parallel performance

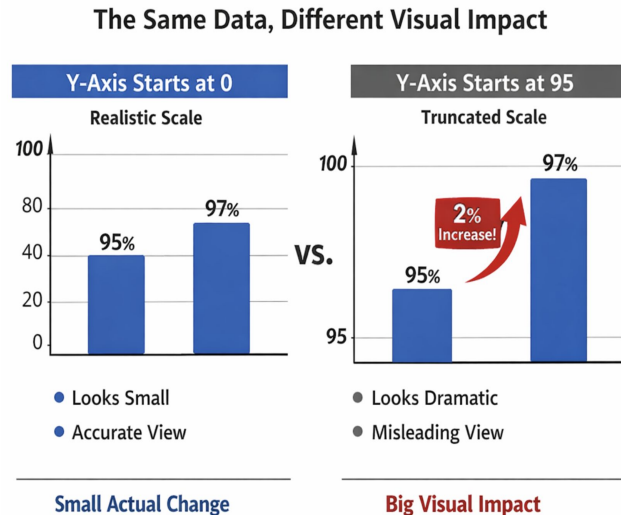
This paper addresses problems in the presentation of parallel performance.

Presentations often:

- Exaggerate or diminish performance differences
- Conceal important Performance trends
- Ignore significant computational costs
- Introduce distortion unintentionally

Goal:

- Present information
- Use Space/time effectively
- Minimize distortion



Measure Elapsed Time

Single process CPU time may be misleading.

CPU time measures only active computation
Elapsed time measures total time to solution

Elapsed time includes:

- Computation
- Lock waiting
- Synchronization
- Cache coherence overhead
- Paging and I/O
- Operating system scheduling

Parallel execution time is not deterministic.

- Measure multiple runs
- Report mean and variance
- High Variance may indicate contention

Then:

Cache Measurement strategies:

- Measure Stable cache behavior
- Measure transient Caches

Present Elapsed Time

Present elapsed time.

Avoid presenting speedup where possible because:

- Speedup is a derivative measure
- It depends on the chosen sequential baseline
- Different derivations can lead to different conclusions

Present elapsed time directly to avoid undocumented derivations.

Compare

Parallel performance depends on many independent variables.

- Algorithms
- Parallelization
- Programming System
- Machine architecture
- Problem Size
- Number of processors

To avoid distortion:

- Change as few independent variables as possible
- Document all changes clearly
- Compare systems under controlled conditions

CPU Time vs Elapsed Time

CPU time and elapsed time behave differently in parallel systems.

CPU time often decreases smoothly with processors.

Elapsed time behaves differently.

Parallel coordination overhead includes:

- Lock contention
- Synchronization barriers
- Cache Coherence Traffic
- Paging delays
- OS scheduling

Elapsed time reflects true system-level performance

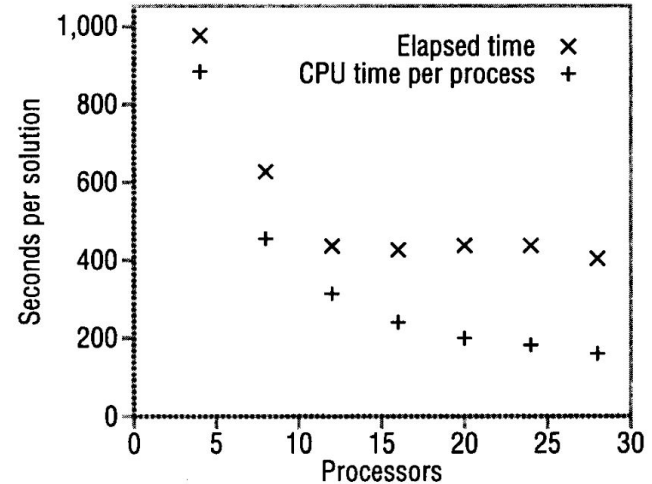


Figure 2. Elapsed time versus average CPU time per process.¹

Variance Predicts Scalability Instability

Growing variance signals:

- Contention amplification
- Synchronization pressure
- Resource interference
- Lock convoying

Then:

- Execution time variance must be reported.
- High Variance may indicate structural scalability problems

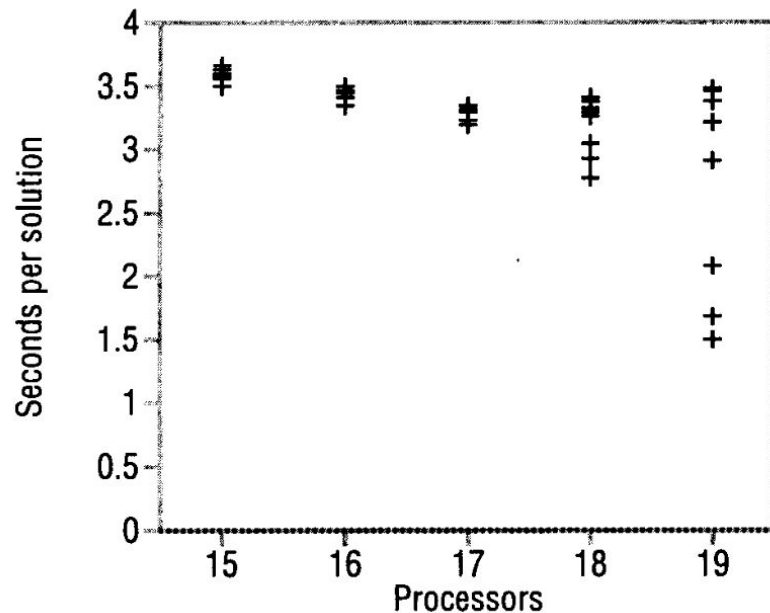


Figure 1. Variation when measuring single-process CPU time.

Log-Log Time Reveals True Scaling Geometry

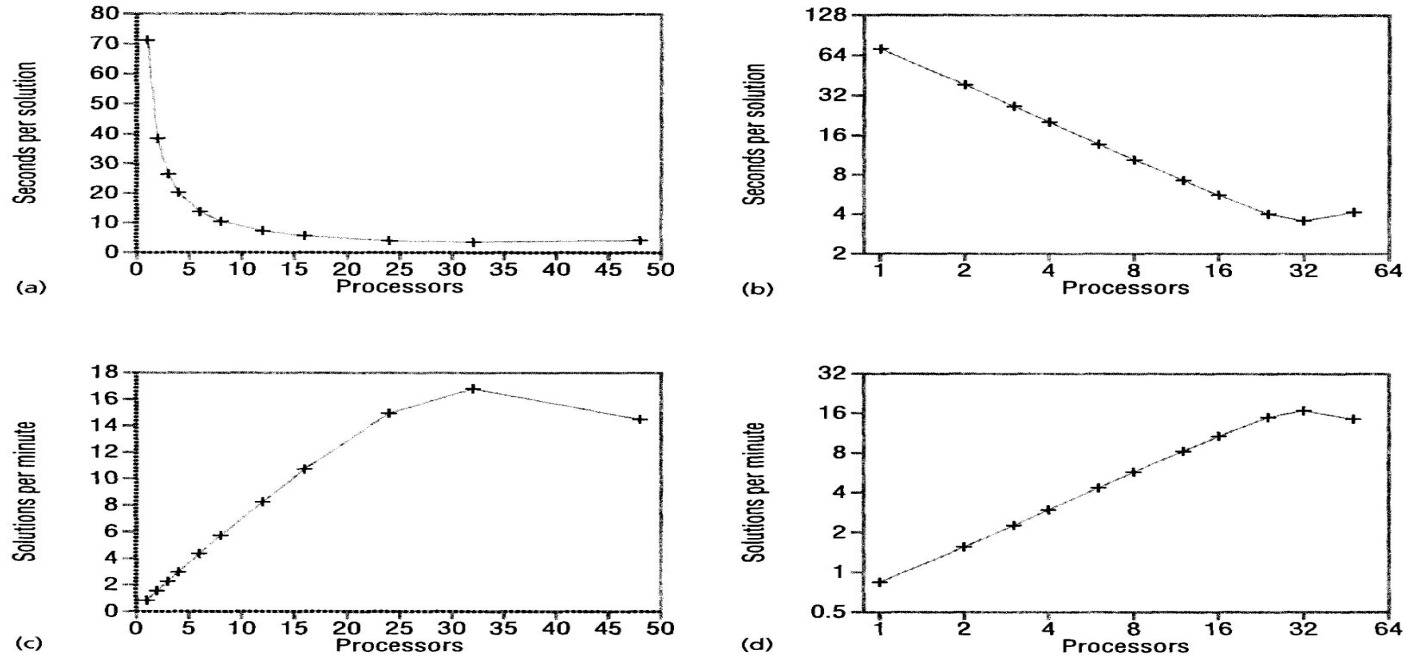


Figure 4. Showing qualitative changes: (a) linear time, (b) log-log time, (c) linear speed, (d) log-log speed.⁷

Linear time plots:

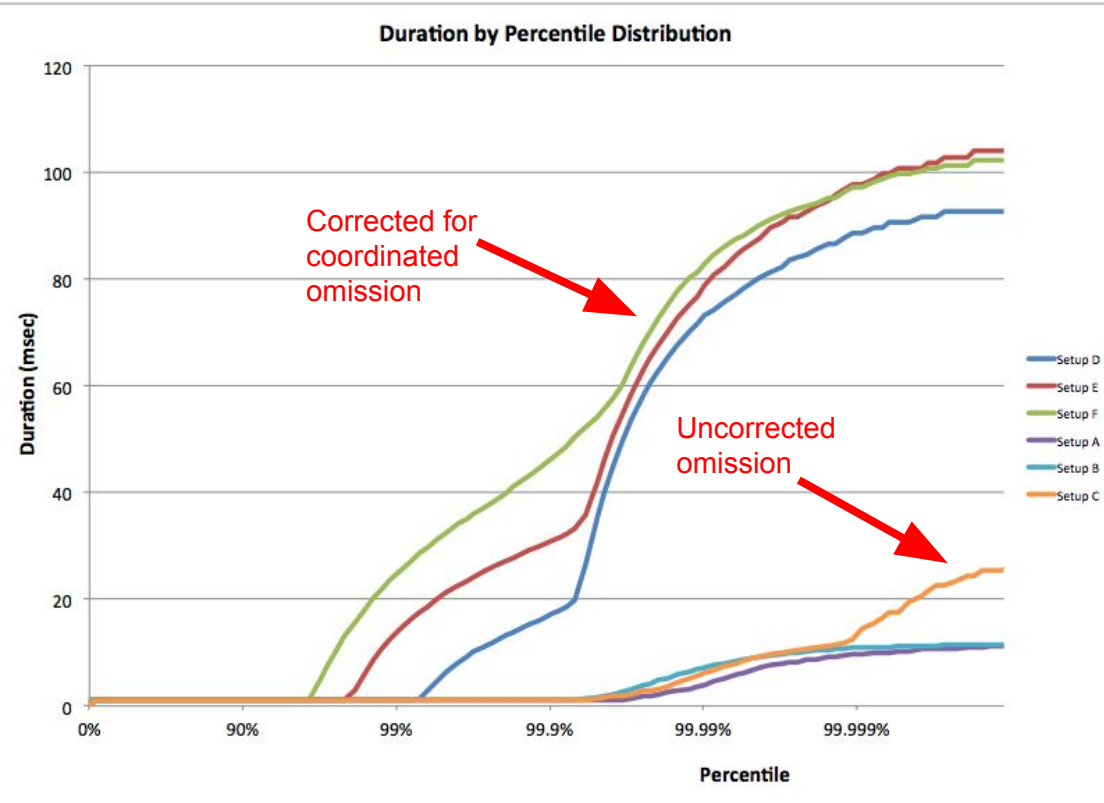
- Hide qualitative changes
- Compress high processor counts
- Make linear scaling appear nonlinear

Log-Log time plots:

- Make linear performance appear linear
- Reveal qualitative changes
- Preserve Relative performance
- Scale Invariant

Use log-log time plots when absolute performance is not the primary concern.

benchmarking crime #1 - Gil Tene's coordinated omission



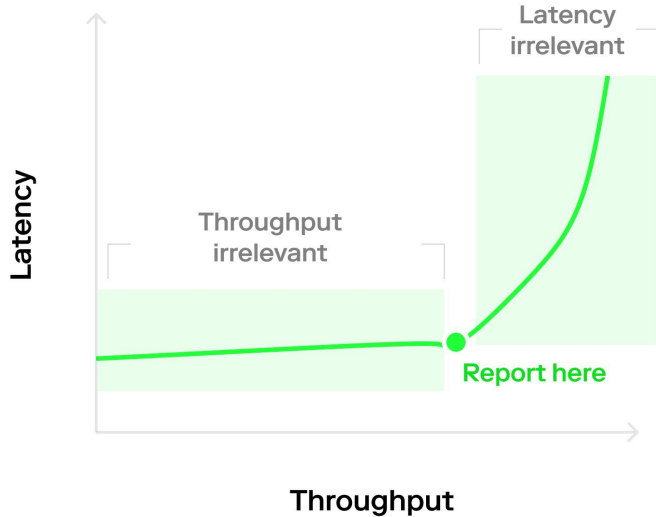
illusion (Blue Line): Standard "Closed Loop" tools (e.g., [ab](#), JMeter) report low latency even at high percentiles because they pause during server stalls.

reality (Red Line): "Open Loop" testing corrects this by measuring the *actual* wait time during stalls.

delta: The massive gap between the curves represents **Survivor Bias**: data points that were "omitted" because the benchmark tool was waiting for a response.

fix: Never trust high-percentile latency (p99, p99.9) from a closed-loop generator.

benchmarking crime #2 - "zero-cost" throughput

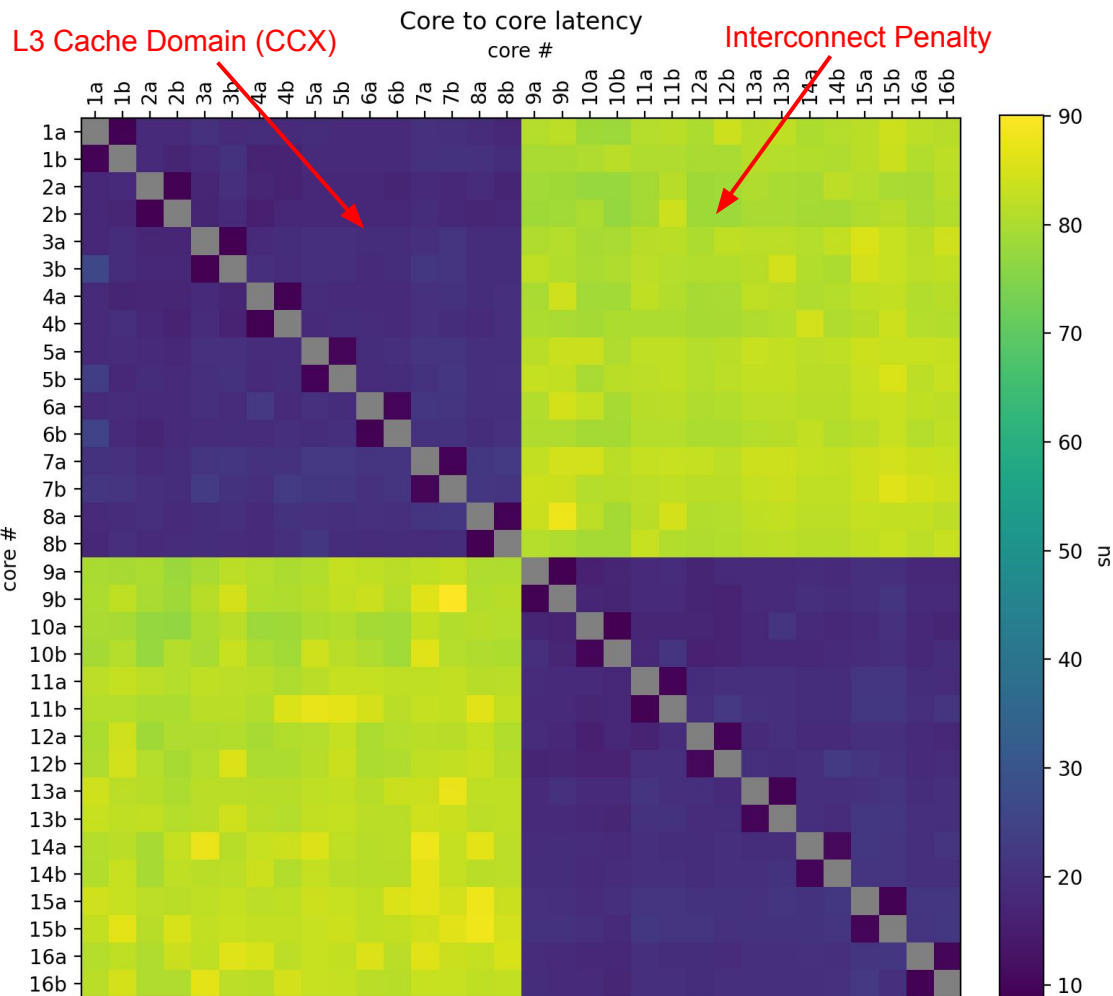


decoupling fallacy: Reporting "Max Throughput" and "Min Latency" as independent metrics

physics: Little's Law ($L = \lambda W$) dictates that as utilization approaches 100%, queue lengths (and latency) approach infinity

saturation point: High throughput is often achieved by buffering, which inherently destroys latency

fix: Never report single numbers. We plot the **Saturation Curve** to identify the "Knee", which is the precise load where latency degrades non-linearly



failure of linearity: High-dimensional data (e.g., $N * N$ core interactions) cannot be flattened into line plots without losing topological context.

"block diagonal" structure:

- **Small Blocks (2 * 2):** Reveal **SMT** (Hyperthreading) siblings sharing L1/L2 cache.
- **Large Blocks (4 * 4):** Reveal **CCX** (Core Complex) boundaries sharing L3 cache.

"off-diagonal" penalty: Yellow zones represent the latency cost of crossing the **infinity fabric** or Interconnect.

This data dictates **thread affinity** (`sched_setaffinity`). We pin communicating threads within the same "Purple Block" to avoid the "Yellow" latency penalty.